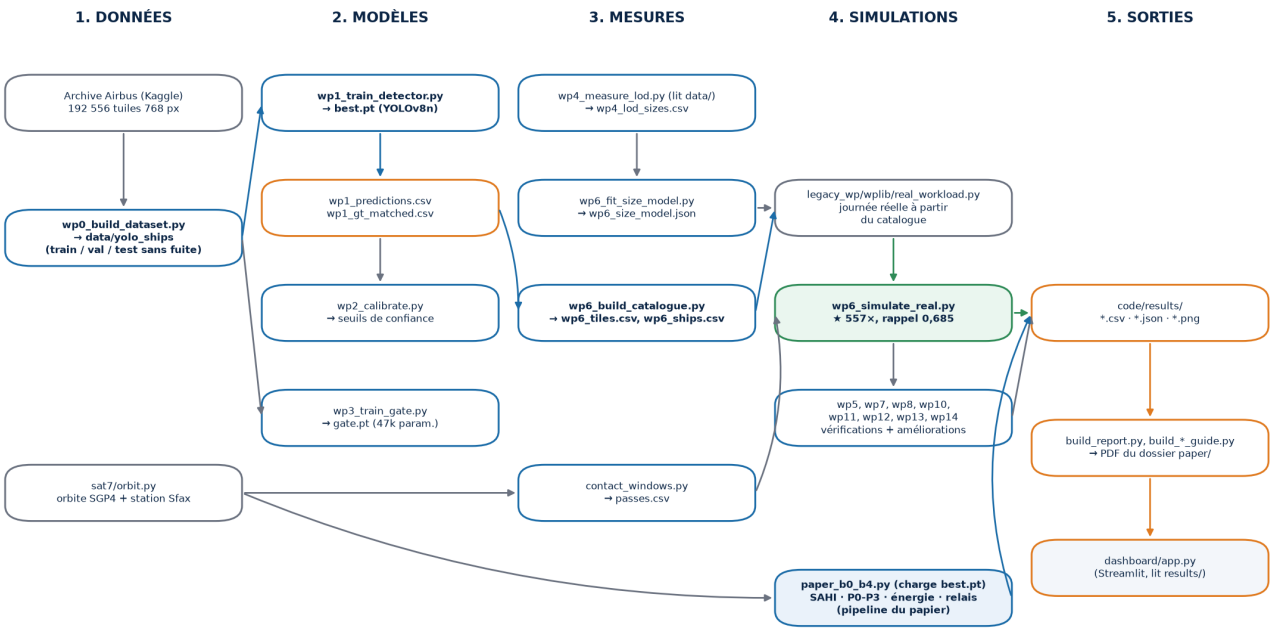
Le pipeline complet

Transmettre de l'information plutôt que des images brutes

IASTAM 6.0 · Track 4 · Problème 7 · ENIS



en gras : les étapes clés · vert : le résultat principal mesuré · bleu clair : le pipeline du papier · sat7/ = la bibliothèque partagée

La carte du projet entier : des données Airbus jusqu'aux rapports, avec le pipeline mesuré (WP) et le pipeline du papier.

Ce document explique tout le projet, de zéro : l'idée, chaque étape, chaque fichier, comment les fichiers se passent les données, comment lire ce que le terminal affiche, les résultats, et comment tout relancer.

Version du 09/10/2026

Sommaire

- 1. Le problème, en mots simples
- 2. Le vocabulaire à connaître
- 3. La carte du projet : deux pipelines, une bibliothèque
- 4. Le pipeline mesuré (WP) : ce qui se passe à bord
- 5. Le pipeline mesuré, script par script (wp0 à wp14)
- 6. Le pipeline du papier (SAHI, P0-P3, énergie, relais)
- 7. La bibliothèque sat7/ : chaque module
- 8. Qui produit quoi, qui lit quoi : le flux des fichiers
- 9. Lire les sorties du terminal
- 10. Les résultats
- 11. Les tests
- 12. Tout relancer : environnements et commandes
- 13. Ce qui est mesuré, ce qui est supposé, les limites
- 14. Le tableau de bord
- 15. La fusion avec le dépôt de l'ami (oblhdj)
- Annexe : l'organisation des dossiers

1. Le problème, en mots simples

Un petit satellite d'observation photographie la mer en continu. Il produit beaucoup plus d'images qu'il ne peut en envoyer à la Terre, pour deux raisons :

- **Il ne peut parler à la station sol que quand il passe au-dessus d'elle.** Pour la station de Sfax (ENIS), le calcul d'orbite donne 5 passages par 24 h, de 3 à 9 minutes chacun, soit 34 minutes de contact par jour.
- **Le débit est limité** : environ 649 Mo par jour au total avec une radio S-band de CubeSat. Et pendant 11,4 heures d'affilée, il ne peut rien envoyer du tout.
- Une journée d'images brutes représente environ **60 000 Mo**. Il faudrait donc près de 100 fois plus de liaison que ce qui existe.

La solution classique consiste à compresser davantage les images. **Notre solution** est de ne plus envoyer les images, mais **ce que le satellite y a vu** :

- un détecteur de navires tourne à bord ;
- un navire dont on est sûr devient une ligne de texte de 40 octets (position, taille, confiance) ;
- un navire douteux est envoyé avec une petite image (imagette) pour vérification ;
- les zones côtières, où le détecteur se trompe souvent, sont envoyées entières ;
- à chaque passage, on envoie d'abord ce qui a **le plus de valeur par octet**.

Résultat mesuré sur de vraies images Airbus : 557 fois moins de données qu'en envoyant les images brutes, et 68,5 % des navires arrivent au sol (entre 40 % et 82 % selon la couverture nuageuse supposée). En envoyant les images brutes, seuls 0,2 % arriveraient, car la liaison serait saturée.

2. Le vocabulaire à connaître

Terme	Signification
Tuile	Un morceau d'image carré (768 × 768 pixels dans Airbus, 512 × 512 dans le papier).
Détecteur (YOLO)	Réseau de neurones qui dessine une boîte autour de chaque navire et donne un score de confiance entre 0 et 1. Nous utilisons YOLOv8n, la plus petite version.
Confiance	Ce score. Au-dessus d'un seuil (0,25 par défaut), la boîte est gardée.
Rappel (recall)	Part des vrais navires trouvés. 0,685 = 68,5 % des navires.
Précision	Part des détections qui sont de vrais navires (le reste = fausses alarmes).
F1	Moyenne harmonique de la précision et du rappel.
mAP50	Qualité globale d'un détecteur : précision moyenne quand une boîte doit recouvrir le vrai navire à au moins 50 % (IoU ≥ 0,5).
IoU	Intersection sur union : à quel point deux boîtes se recouvrent (0 à 1).
Gate	Petit réseau (47 000 paramètres) qui dit très vite si une tuile peut contenir un navire. Les tuiles de mer vide ne passent pas au détecteur.
Niveau de détail (LoD)	Combien on envoie pour un navire : 40 octets de texte, une imagette (~900 o), une zone (~2,5 ko) ou la tuile entière.
Passage	Moment où le satellite est visible depuis la station sol.
Downlink	Liaison descendante satellite → sol.
Bent pipe	« Tuyau coudé » : on renvoie tout, brut, sans traitement à bord.
FIFO	Premier arrivé, premier envoyé.
Value-greedy	Notre ordonnanceur : envoie d'abord ce qui a le plus de valeur par octet.
Sac à dos (knapsack)	Problème mathématique : remplir une capacité limitée avec les objets les plus précieux. Chaque passage est un sac à dos.
SAHI	Découper une grande image en tuiles qui se chevauchent, détecter sur chacune, puis recoller. Aide pour les petits objets.
Fusion (NMS / NMM)	Supprimer ou fusionner les doublons quand un navire est vu dans deux tuiles voisines.
P0 à P3	Les priorités du papier : rien, métadonnées, + ROI, + image de contexte.
ROI	Région d'intérêt : petite image découpée autour du navire.
ISL	Liaison inter-satellite : un second satellite relaie les données.
SGP4	Le modèle standard qui calcule la position d'un satellite dans le temps.
REAL / SIM	REAL = mesuré sur de vraies images ; SIM = simulé. Chaque chiffre porte son étiquette.

3. La carte du projet : deux pipelines, une bibliothèque

Le dossier code/ est organisé comme le papier :

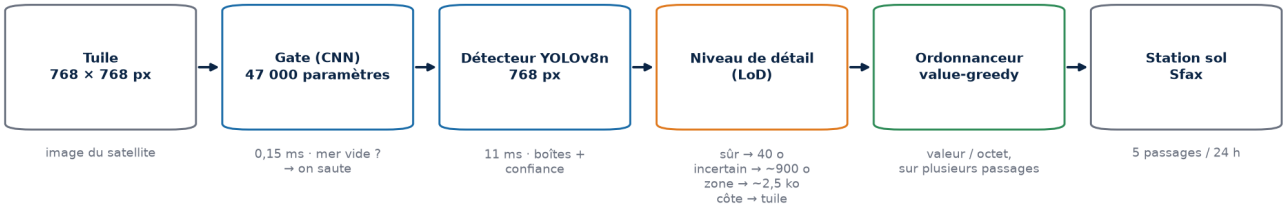
- `sat7/` : la **bibliothèque du papier**, 15 modules : SAHI, fusion, objectif joint, enregistrements sémantiques, énergie, relais, métriques, plus le socle partagé (orbite, doublons, filtre classique). Aucun module ne lance d'expérience tout seul.
- `scripts/` : la campagne du papier (`paper_b0_b4.py`), le dataset (wp0), le détecteur (wp1), sa calibration (wp2), les passages et les guides.
- `dashboard/` : le tableau de bord, en lecture seule.
- `legacy_wp/` : le **pipeline WP archivé**, toujours exécutable : ses modules (`wplib/` : ordonnanceur, optimum, journée réelle), ses scripts wp3 à wp14 et ses tests. C'est lui qui a produit les résultats mesurés sur Airbus.
- `results/` : tout ce que les scripts produisent (CSV, JSON, PNG) ; les journaux d'exécution sont dans `results/logs/`. Les rapports et les PDF lisent ces fichiers, ils ne recalculent rien.

Deux pipelines utilisent cette bibliothèque :

	Pipeline mesuré (WP)	Pipeline du papier
Lancé par	wp0 → wp14 (un script par lot de travail)	paper_b0_b4.py
Images	tuiles Airbus 768 px, une par une	grande image découpée en tuiles 512 px avec 20 % de chevauchement (SAHI)
À bord	gate → YOLO → niveau de détail (L0/L1/L2/tuile)	SAHI + YOLO → fusion → enregistrements P0-P3
Transmission	ordonnanceur value-greedy sur plusieurs passages	liaison directe ou relais inter-satellite, choisi par énergie et latence
Évalue	navires livrés, octets, latence, optimalité	+ énergie, préservation de l'information, B0-B4, ablations
Chiffres	REAL : mesurés sur Airbus	SIM pour l'instant ; REAL après un lancement avec YOLO sur la machine GPU

Comment ils se complètent. Le pipeline WP a construit et mesuré toutes les briques réelles : le dataset, le détecteur entraîné, le coût réel en octets, l'orbite. Le pipeline du papier implémente l'architecture décrite dans l'article de l'équipe (SAHI, énergie, relais) en réutilisant ces briques.

4. Le pipeline mesuré (WP) : ce qui se passe à bord



La chaîne à bord du pipeline mesuré, pour chaque tuile.

Étape 1 — Le gate : ne pas regarder la mer vide

La plupart des tuiles ne contiennent que de l'eau. Un petit réseau de 46 897 paramètres regarde une version réduite de la tuile (128 px) et décide si elle peut contenir un navire. Au seuil choisi, il garde 99.0% des navires et écarte 36% des tuiles vides, en 0.052 ms par tuile sur GPU. Le filtre classique (Otsu, morphologie) testé avant ne gardait que 64.5% des navires : c'est un résultat négatif que le projet assume.

Étape 2 — Le détecteur YOLOv8n

Il trouve les navires et donne une confiance. Sur le jeu de test : mAP50 0,804, précision 0,817, en 11 ms par tuile sur GPU. Il rate surtout les petits navires : rappel 0,739 sous 32 px, 0,958 entre 32 et 96 px, 0,994 au-dessus.

Étape 3 — Le niveau de détail (LoD)

Pour chaque détection, on choisit combien envoyer selon la confiance :

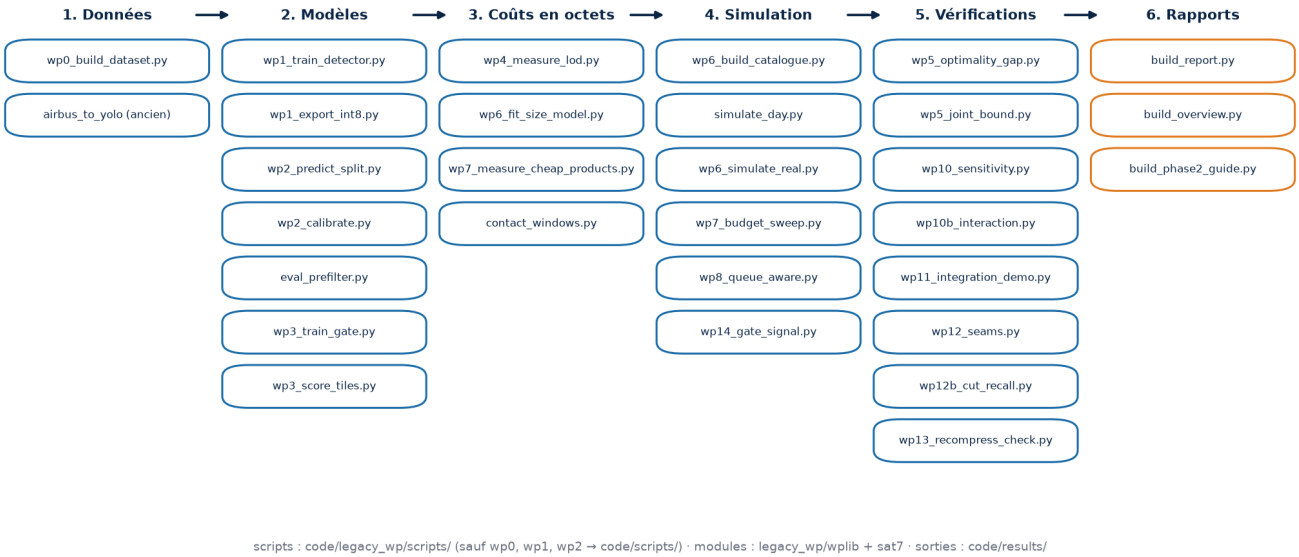
Niveau	Contenu	Taille	Quand
L0	métadonnées : position, longueur, cap, confiance	40 octets	détection très sûre
L1	imagette du navire	~900 octets (selon la taille du navire)	détection plausible
L2	zone autour du navire	~2,5 ko	détection douteuse
Tuile	tuile côtière entière (JPEG)	~31 ko (q40)	côte : le détecteur peut avoir raté un navire
Vignette	aperçu minuscule de chaque tuile	quelques centaines d'octets	filet de sécurité

Étape 4 — L'ordonnanceur value-greedy

Tous ces objets attendent dans la mémoire du satellite. À chaque passage au-dessus de Sfax, la capacité est limitée : c'est un problème de sac à dos. L'ordonnanceur trie les objets par **valeur divisée par la taille** et remplit le passage dans cet ordre. Si un objet ne rentre pas en entier, il peut être **tronqué progressivement** (on envoie le début d'une image progressive). Les objets non envoyés attendent le passage suivant.

Pourquoi ne pas faire FIFO ? Tant que la liaison suffit, FIFO et value-greedy donnent le même résultat. Quand elle sature (160 000 tuiles par jour), value-greedy livre 2,26 fois plus de navires, car il ne gaspille pas la capacité sur des objets peu utiles.

5. Le pipeline mesuré, script par script



Les scripts WP, dans l'ordre où leurs sorties s'enchaînent.

Où sont ces scripts. Les phases 1 et 2 jusqu'à wp2 (dataset, détecteur, calibration) et contact_windows sont dans `code/scripts/` : le pipeline du papier en a besoin. Tous les autres sont archivés dans `code/legacy_wp/scripts/`, inchangés : ils lisent et écrivent toujours code/results/.

Pour chaque script : à quoi il sert, ce qu'il lit, ce qu'il écrit dans results/. « GPU » signifie qu'il faut l'environnement .venv312 avec PyTorch.

Phase 1 — Construire le dataset

Script	Ce qu'il fait	Lit	Écrit
wp0_build_dataset.py	Lit l'archive Airbus directement dans le zip, trouve les tuiles presque identiques (sat7/dedup) et répartit des **groupes entiers** entre train, val et test : aucun navire ne peut apparaître des deux côtés.	archive .zip	data/yolo_ships/, split.csv, wp0_stats.json, wp0_groups.png
airbus_to_yolo.py	Ancienne conversion Airbus → YOLO, remplacée par wp0 (elle ne détectait pas les doublons).	csv Airbus	data/…

Phase 2 — Les modèles (GPU)

Script	Ce qu'il fait	Lit	Écrit
wp1_train_detector.py	Entraîne YOLOv8n 12 époques à 768 px, mesure mAP50 et le rappel par taille, écrit toutes les prédictions ≥ 0,05.	data/yolo_ships	best.pt, wp1_metrics.json, wp1_predictions.csv, wp1_gt_matched.csv
wp1_export_int8.py	Exporte en ONNX FP32 et INT8 et chronomètre. Résultat négatif : INT8 est 8,5× plus lent sur CPU.	best.pt	wp1_export.json
wp2_predict_split.py	Prédictions sur la partie val, jamais utilisée pour le test, afin de calibrer sans tricher.	best.pt	wp2_val_predictions.csv
wp2_calibrate.py	Vérifie si la confiance est une vraie probabilité et recale les seuils du LoD.	wp1/wp2 predictions	wp2_calibration.json, wp2_thresholds.csv
eval_prefilter.py	Teste le filtre classique comme gate : 0,645 de rappel, insuffisant.	images	prefilter_*.csv
wp3_train_gate.py	Entraîne le gate de 47 000 paramètres et trace le compromis rappel / tuiles écartées.	data/yolo_ships	gate.pt, wp3_gate.json
wp3_score_tiles.py	Donne un score de gate à chaque tuile du catalogue.	gate.pt	wp3_gate_scores_test.csv

Phase 3 — Mesurer les coûts

Script	Ce qu'il fait	Lit	Écrit
wp4_measure_lod.py	Découpe de vrais navires, les encode en JPEG à plusieurs qualités et mesure les octets réels de chaque niveau.	data/yolo_ships	wp4_lod_sizes.csv
wp6_fit_size_model.py	Ajuste octets = a × (taille du navire)^b : un cargo coûte plus qu'un bateau de pêche.	wp4_lod_sizes.csv	wp6_size_model.json
wp7_measure_cheap_products.py	Mesure des versions moins chères des vignettes et des tuiles côtières.	images	wp7_cheap_products.csv
contact_windows.py	Calcule les passages au-dessus de Sfax et la capacité de chacun.	sat7/orbit.py	passes.csv, passes.png

Phase 4 — Simuler une journée

Script	Ce qu'il fait	Lit	Écrit
wp6_build_catalogue.py	Transforme les 5 320 tuiles de test et les vraies détections en catalogue : contexte, navires, confiance, fausses alarmes.	wp1_predictions.csv, wp1_gt_matched.csv	wp6_tiles.csv, wp6_ships.csv
simulate_day.py	Première expérience, sur une journée inventée (SIM).	—	sim_*.csv
wp6_simulate_real.py	**Le résultat principal** : la même expérience sur une journée construite à partir du catalogue réel. 557×, rappel 0,685.	wp6_tiles/ships.csv, wp6_size_model.json	wp6_real_table.csv, wp6_thr_sweep.csv…
wp7_budget_sweep.py	Où partent les octets ; compare q60, q40, q30 et la mosaïque.	catalogue	wp7_budget_sweep.csv
wp8_queue_aware.py	Le satellite regarde sa file d'attente pour décider combien de détail envoyer pour la côte.	catalogue	wp8_queue_aware.csv
wp14_gate_signal.py	Remplace le filtre classique par le gate appris pour décider des vignettes.	wp3_gate_scores_test.csv	wp14_gate_signal.csv

Phase 5 — Vérifier

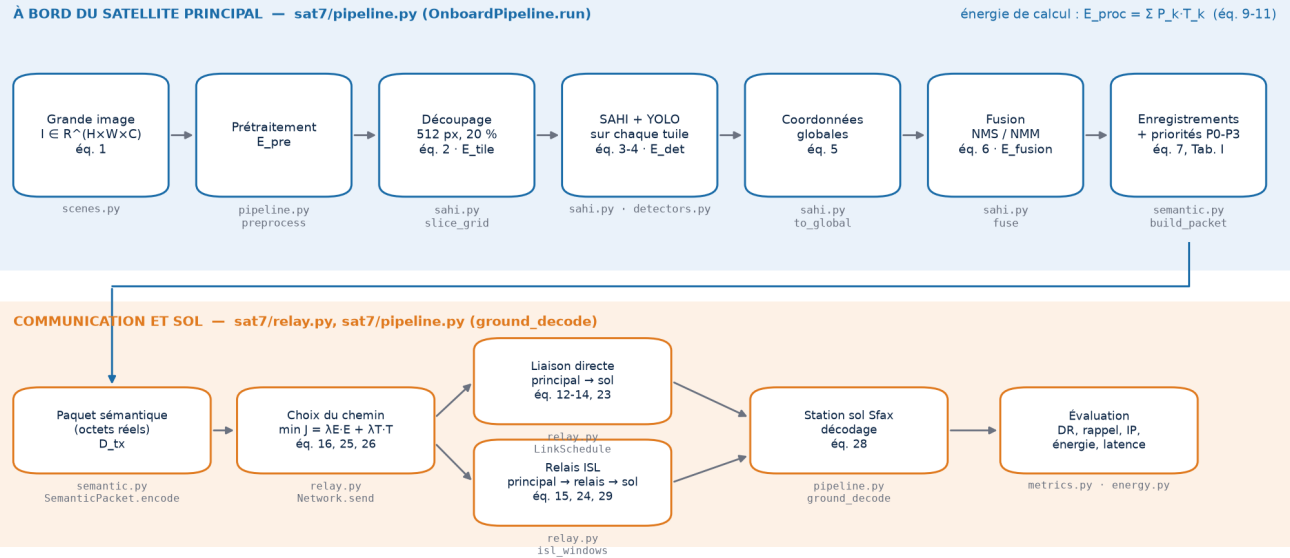
Script	Ce qu'il fait	Lit	Écrit
wp5_optimality_gap.py	Compare value-greedy à l'optimum exact (programmation dynamique, sat7/optimum) à chaque passage.	catalogue	wp5_optimality_gap.csv
wp5_joint_bound.py	Borne la perte sur la journée entière, pas seulement par passage.	catalogue	wp5_joint_bound.csv
wp10_sensitivity.py	Fait varier chaque constante inventée et regarde si les conclusions tiennent.	catalogue	wp10_sensitivity.csv, wp10_tornado.png
wp10b_interaction.py	Fait varier deux constantes à la fois.	catalogue	wp10b_interaction.csv
wp11_integration_demo.py	Fait passer de vraies images JPEG dans toute la chaîne et vérifie qu'on retrouve le catalogue à 4 décimales près (GPU).	images, best.pt, gate.pt	wp11_integration.json
wp12_seams.py, wp12b_cut_recall.py	Mesurent ce qui arrive aux navires coupés par un bord de tuile et ce que SAHI rattrape (GPU).	best.pt	wp12_*.csv/json
wp13_recompress_check.py	Vérifie si la compression q30 abîme la détection au sol : oui, −4 points sur les petits navires.	best.pt	wp13_recompress.json

Phase 6 — Rapports

Script	Ce qu'il fait	Lit	Écrit
build_report.py	Rapport technique PDF, chiffres lus dans results/.	results/	PDF
build_overview.py	Vue d'ensemble PDF.	results/	PDF
build_phase2_guide.py	Guide du poster PDF.	results/	PDF
guide_figures.py	Toutes les figures de ce document.	results/paper_*	results/guide/*.png
build_pipeline_guide.py, build_full_guide.py	Les deux guides en français.	results/	PDF

6. Le pipeline du papier

L'article de l'équipe, *Energy-Aware Semantic Downlink for Maritime Object Detection with Optional Inter-Satellite Relay*, décrit une architecture un peu différente. Elle est implémentée dans `sat7/pipeline.py` et lancée par `scripts/paper_b0_b4.py`.



Le pipeline du papier : en haut à bord, en bas communication et sol. Sous chaque étape : le fichier et la fonction.

Ce qui change par rapport au pipeline WP

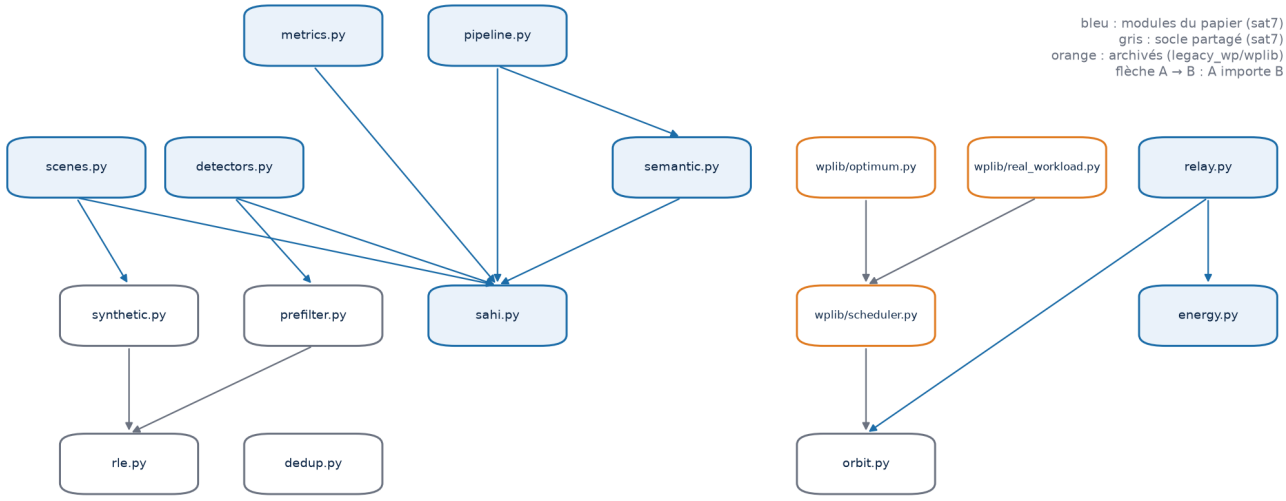
- **Grande image et SAHI** : au lieu d'une tuile, le satellite reçoit une grande scène. Elle est découpée en tuiles de 512 px qui se chevauchent de 20 %, pour qu'un navire au bord d'une tuile soit entier dans la voisine (éq. 1-4).
- **Coordonnées globales** : une boîte trouvée dans une tuile est ramenée dans le repère de la grande image : x_g = x_0 + x_l (éq. 5).
- **Fusion** : un navire vu dans deux tuiles voisines donne deux boîtes ; la fusion n'en garde qu'une (éq. 6). Sans elle, la précision chute de 0,63 à 0,40 en simulation.
- **Enregistrement sémantique et P0-P3** : chaque détection devient un enregistrement binaire de 24 octets (classe, confiance, boîte, horodatage, id image). Plus la confiance est basse, plus on joint d'images : P1 seul, P2 avec ROI, P3 avec ROI et contexte.
- **Énergie** : chaque étape est chronométrée et multipliée par une puissance supposée (E = P × T) ; la transmission coûte P_tx × octets / débit (éq. 8-14).
- **Relais inter-satellite** : si un second satellite peut transmettre plus tôt, le paquet passe par lui. Le choix minimise J = λ_E·E + λ_T·T sous une échéance (éq. 16, 25, 26).

La campagne B0 → B4

Config.	À bord	Envoyé	Mesure
B0	rien	image brute	référence
B1	YOLO sur l'image entière	détections	réduction par le détecteur seul
B2	SAHI + YOLO	détections	apport de SAHI
B3	SAHI + YOLO + politique	métadonnées / ROI, direct	apport sémantique
B4	SAHI + YOLO + politique	direct ou relais	stratégie complète

Plus 6 ablations (sans SAHI, sans chevauchement, sans fusion, sans politique adaptative, sans ROI, sans relais) et les courbes de compromis des éq. 32 à 35. Le détail fichier par fichier est dans le *Guide du pipeline* (IASTAM_P7_Guide_Pipeline_FR.pdf).

7. La bibliothèque sat7/ : chaque module



Qui importe qui dans sat7/. Extrait des imports réels du code.

Module	Rôle	Contenu principal	Utilisé par
rle.py	Format Airbus	rle_decode/encode (masques compressés), Box, mask_to_box	prefilter, synthetic, wp0
dedup.py	Doublons du dataset	phash, signatures, candidate_pairs, UnionFind, build_groups, group_aware_split, check_no_leakage	wp0
prefilter.py	Filtre classique de vision	run_prefilter (HSV, top-hat, Otsu, watershed, Canny) → nuage / côte / mer vide / candidats	eval_prefilter, wp6, wp11, detectors
synthetic.py	Fausses tuiles pour les tests	make_tile, draw_ship	tests, scenes
orbit.py	Orbite et liaison	make_satellite (SGP4), find_passes, rate_bps (Shannon), LINK_PRESETS	scheduler, relay, contact_windows
legacy_wp/wplib/scheduler.py	Cœur du pipeline WP	Item, Ship, encode_raw / fixed_patch / lod / tile, politiques FIFO / NoBuffer / ValueGreedy, simulate, metrics	wp5-wp14
legacy_wp/wplib/optimum.py	Optimum exact	best_value (programmation dynamique), fractional_bound, joint_upper_bound, measure_gap	wp5
legacy_wp/wplib/real_workload.py	Journée réelle	load_catalogue, load_size_model, workload_from_catalogue	wp5-wp14
sahi.py	Papier éq. 1-6	Detection, SahiConfig, slice_grid, fuse, sahi_detect	pipeline, detectors, semantic, metrics, scenes
detectors.py	Brancher un détecteur	YoloDetector, ClassicDetector (secours)	paper_b0_b4
semantic.py	Papier éq. 7, Tableau I	SemanticRecord, PriorityPolicy, build_packet, SemanticPacket.encode/decode	pipeline
pipeline.py	Chaîne de bord du papier	OnboardPipeline, PipelineConfig, ground_decode	paper_b0_b4
energy.py	Papier éq. 8-22, 27-30	PowerConfig, processing_energy, comm_energy_direct/relay, energy_saving, select_operating_point	relay, paper_b0_b4
relay.py	Papier éq. 15-16, 23-26	LinkSchedule, ground_windows, isl_windows, Network.send	paper_b0_b4
metrics.py	Papier §VIII-E	DetAccumulator (P, R, F1, AP), preserved	paper_b0_b4
scenes.py	Grandes scènes	swaths_from_manifest (24 bandes réelles), airbus_mosaic, synthetic_scene	paper_b0_b4

Module	Rôle	Contenu principal	Utilisé par
joint.py	Papier éq. 20-22 (fusion)	solve (niveau P0-P3 par objet : borne inférieure + réparation), pareto, constraint_sweep, encode_candidates	paper_b0_b4
datasets.py	Papier §VIII-C (fusion)	parse_dota_lines, coco_ground_truth, iter_dota, iter_coco (DOTA, HRSID)	paper_b0_b4

8. Qui produit quoi, qui lit quoi

Les scripts ne s'appellent presque jamais entre eux : **ils communiquent par fichiers**. Un script écrit un fichier dans results/ (ou data/, runs/, models/), un autre le lit. C'est ce qui permet de relancer une seule étape sans tout refaire. Les fichiers clés :

Fichier	Produit par	Lu par	Contenu
data/yolo_ships/	wp0	wp1, wp2, wp3, wp4, wp7, wp11, wp12, paper_b0_b4 (airbus)	images + étiquettes train / val / test
runs/…/best.pt	wp1	wp1_export, wp2, wp11, wp12, wp13, paper_b0_b4 (yolo)	poids du détecteur
models/gate.pt	wp3_train_gate	wp3_score_tiles, wp11	poids du gate
wp1_predictions.csv	wp1	wp2_calibrate, wp6_build_catalogue	chaque boîte prédite avec sa confiance
wp1_gt_matched.csv	wp1	wp6_build_catalogue	chaque vrai navire : trouvé ? confiance ?
wp4_lod_sizes.csv	wp4	wp6_fit_size_model	octets réels de chaque niveau
wp6_size_model.json	wp6_fit_size_model	real_workload → wp5, wp6, wp7, wp8, wp10, wp14	loi octets = a·taille^b
wp6_tiles.csv, wp6_ships.csv	wp6_build_catalogue	real_workload → wp5-wp14, wp11, wp13	le catalogue réel des 5 320 tuiles de test
wp3_gate_scores_test.csv	wp3_score_tiles	wp14	score du gate par tuile
passes.csv	contact_windows	rapports	passages au-dessus de Sfax
paper_*_sim.csv / .json	paper_b0_b4	guide_figures, guides PDF, tableau de bord	résultats B0-B4, ablations, balayage, paquets
results/*.png	chaque script	PDF du dossier paper/	les graphiques

Pourquoi vos commandes tournent vite. Les étapes lourdes (wp0 à wp4, qui ont besoin des images et du GPU) ont déjà tourné ; leurs fichiers sont dans results/. Les scripts d'analyse (wp5 à wp10, paper_b0_b4 en mode SIM) ne font que relire ces fichiers.

9. Lire les sorties du terminal

Les commandes se lancent depuis le dossier code/. Voici ce que chacune affiche et comment le lire.

pytest — les tests

`140 passed, 2 skipped` : tout est correct. Les 2 tests ignorés ont besoin de PyTorch, absent de .venv. L'avertissement « DeprecationWarning » vient de la bibliothèque skyfield et ne concerne pas notre code.

contact_windows.py — les passages

Un tableau avec une ligne par passage : heure de lever, heure de coucher, durée, élévation maximale et capacité en Mo. Un passage haut (66,7°) dure plus longtemps et porte plus (240 Mo) qu'un passage rasant (6,2°, 34 Mo). En bas : 34,2 minutes de contact et 649 Mo par jour, et 11,4 h sans contact au maximum.

wp6_simulate_real.py — le résultat principal

- Première partie : la journée simulée (40 000 tuiles, 21 137 navires réels, 6 passages sur 36 h, 210 Mo de capacité).
- Tableau des stratégies : ship_recall = part des navires livrés. Bent pipe 0,002 ; Phi-sat-2 équitable 0,622 ; la nôtre 0,685, avec 108 Mo envoyés.
- REAL vs SYNTHETIC : la même expérience sur données inventées donnait 0,998 ; le vrai chiffre est 0,685. Le tableau est là pour montrer cet écart honnêtement.
- Balayage du seuil de confiance : plus le seuil est bas, plus on trouve de navires mais plus il y a de fausses alarmes (0,05 → 16 576 fausses alarmes ; 0,70 → 200). 0,25 est le compromis retenu.

wp7_budget_sweep.py — où vont les octets

Chaque ligne est une façon d'envoyer les tuiles côtières. q60, q40 et q30 sont des qualités JPEG : moins d'octets, même rappel 0,685. « ROI mosaic only » est très économe mais perd des navires (0,622). « gated » : le gate saute certaines vignettes.

wp8_queue_aware.py — adaptation à la charge

Le tableau « Regret » dit combien chaque politique fixe perd par rapport à la meilleure possible, quand on ne sait pas d'avance si la liaison sera saturée. Plus petit = mieux. Notre politique « queue-aware » a le plus petit regret maximal (0.0058).

wp5_optimality_gap.py --frac-steps 16 — greedy contre optimum

mean_gap et max_gap : écart entre value-greedy et l'optimum exact. Des valeurs comme 2,4e-16 valent zéro (erreur d'arrondi de l'ordinateur) : à charge normale, greedy trouve l'optimum. FIFO, lui, perd jusqu'à 0,86. « saturated » = part des passages où la liaison était pleine.

wp10_sensitivity.py — les hypothèses

« violated in 0 of 80 settings » : nos deux conclusions principales restent vraies dans les 80 réglages testés. Le tableau suivant classe les hypothèses par effet : la couverture nuageuse fait varier le rappel de 0,40 à 0,82, les autres presque pas. C'est pourquoi le rappel est toujours donné comme une fourchette.

paper_b0_b4.py — la campagne du papier

Une ligne par scène, puis les liaisons sur 48 h, puis trois tableaux : B0 → B4, ablations, balayage du seuil. La dernière partie liste les hypothèses. L'étiquette [SIM] ou [REAL] est dans chaque titre.

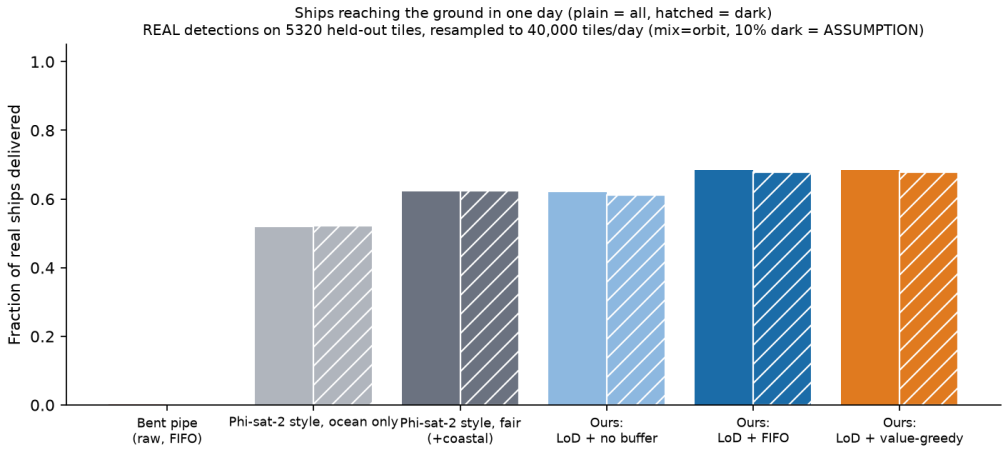
10. Les résultats

Pipeline mesuré [REAL] — vraies tuiles Airbus, vrai détecteur

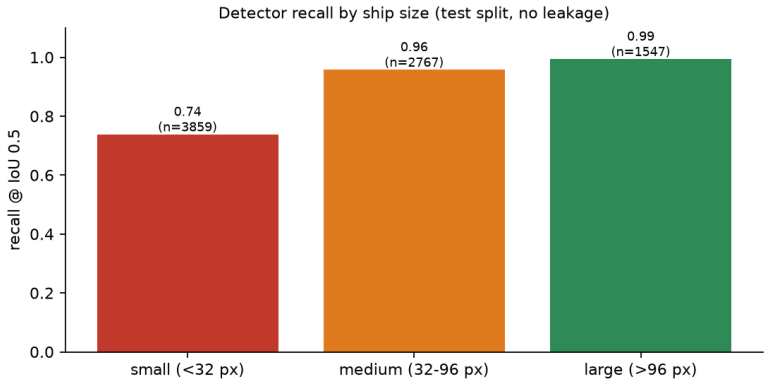
Stratégie	Navires livrés	Plafond	Efficacité liaison
Bent pipe (images brutes)	0,002	0,841	0,003
Phi-sat-2, océan seul	0,518	0,518	1,000
Phi-sat-2 équitable (+ côte)	0,622	0,622	1,000
Nous : LoD sans mémoire	0,620	0,685	0,905
Nous : LoD + FIFO	0,685	0,685	1,000
Nous : LoD + value-greedy	0,685	0,685	1,000

« Plafond » = ce que le satellite a détecté : notre stratégie livre tout ce qu'elle a vu (efficacité 1,000). Le facteur limitant est donc le détecteur, pas la liaison.

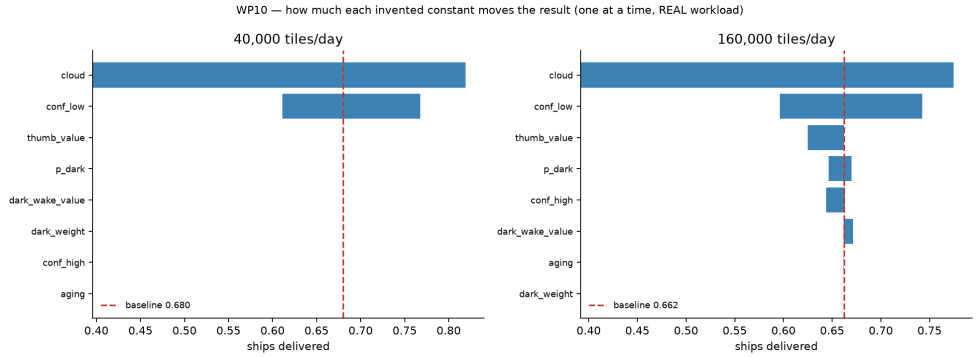
Résultat	Valeur	Comment le lire
Réduction vs images brutes	557×	60 124 Mo offerts → 108 Mo envoyés par jour
Navires livrés	0,685	fourchette 0,40-0,82 selon le nuage supposé (15 % par défaut)
vs FIFO en saturation	2,26×	à 160 000 tuiles/jour
vs Phi-sat-2 équitable	+6,3 points	pour environ 7× plus d'octets
Greedy vs optimum exact	0,251 %	écart, vérifié par programmation dynamique
Détecteur	mAP50 0,804	11 ms par tuile sur GPU



Navires livrés par stratégie (wp6_simulate_real.py).



Rappel du détecteur par taille de navire (wp1).

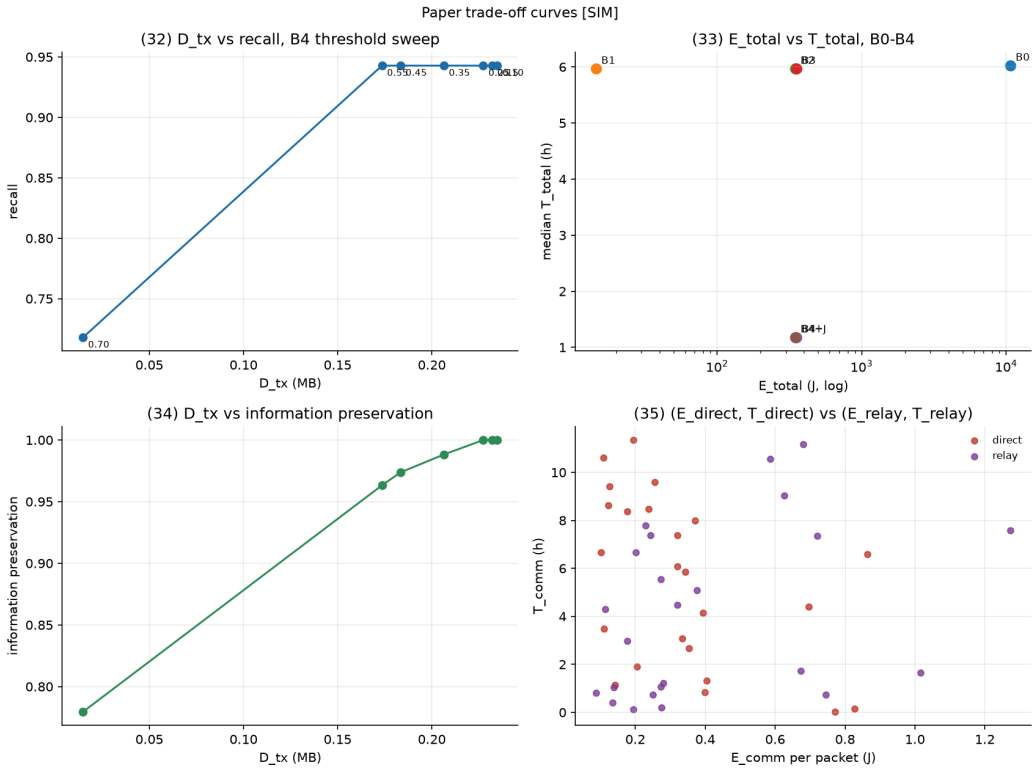


Effet de chaque hypothèse sur le rappel (wp10) : le nuage domine.

Pipeline du papier [SIM] — 24 scènes synthétiques, détecteur de secours

Config.	Mo envoyés	Rappel	IP	Énergie (J)	Latence méd.
B0	302	0,94	1,00	10 733	6,02 h
B1	0.0178	0,67	0,54	15	5,97 h
B2	0.0188	0,94	1,00	348	5,97 h
B3	0.227	0,94	1,00	356	5,97 h
B4	0.227	0,94	1,00	356	1,18 h
B4+J	0.0188	0,94	1,00	348	1,18 h

SAHI fait passer le rappel de 0,67 (B1) à 0,94 (B2) ; le relais divise la latence médiane par environ 5 (B3 → B4). Chiffres SIM : à ne pas citer comme résultats YOLO.



Courbes de compromis du papier, éq. 32-35 [SIM].

11. Les tests

142 tests dans `code/tests/` et `code/legacy_wp/tests/`. Ils vérifient que le code fait ce que les rapports affirment ; si une modification casse une affirmation, un test échoue.

Fichier	Tests	Ce qu'il vérifie
test_rle.py	5	lecture et écriture du format Airbus
test_dedup.py	11	détection des doublons, absence de fuite train/test
test_prefilter.py	5	le filtre classique trouve les navires synthétiques
legacy_wp/tests/test_gate.py	8	les affirmations sur le gate appris (2 ignorés sans torch)
test_calibration.py	7	la calibration de la confiance
test_orbit.py	2	période orbitale, passages, débit décroissant avec la distance
legacy_wp/tests/test_scheduler.py	5	ordonnanceur et politiques
legacy_wp/tests/test_optimum.py	15	l'optimum exact, vérifié contre la force brute
legacy_wp/tests/test_real_workload.py	13	construction de la journée réelle
legacy_wp/tests/test_queue_aware.py	9	la politique qui lit la file d'attente
legacy_wp/tests/test_failure_modes.py	14	injection de pannes (tableau AMDEC du rapport)
test_paper.py	16	une vérification par équation du papier + la chaîne complète
test_dashboard.py	11	chargement des résultats et affichage des 8 pages
test_merge.py	19	la fusion : éq. 20 (unités, bornes), stations multiples, bandes, DOTA/COCO, temps réels du détecteur

12. Tout relancer : environnements et commandes

Environnement	Python	Sert à
.venv	3.14	tout ce qui n'a pas besoin du détecteur : analyses, simulations, tests, paper_b0_b4 en mode SIM
.venv312	3.12 + PyTorch + CUDA	tout ce qui touche au détecteur : wp1, wp2, wp3, wp11, wp12, wp13, paper_b0_b4 avec YOLO
python (système)	3.14 + reportlab	les PDF

Sur cette machine, .venv312, data/ et runs/ ne sont pas présents : les étapes GPU ne peuvent pas tourner ici. Elles ont tourné sur la machine d'entraînement, et leurs sorties sont dans results/.

L'ordre complet (depuis code/)

```
# 0. tests
.venv/Scripts/python.exe -m pytest tests legacy_wp/tests -q

# 1-3. données et modèles (GPU, une seule fois)
.venv312/Scripts/python.exe scripts/wp0_build_dataset.py --zip <archive Airbus>
.venv312/Scripts/python.exe scripts/wp1_train_detector.py --epochs 12 --imgsz 768
.venv312/Scripts/python.exe scripts/wp2_predict_split.py --split val
.venv312/Scripts/python.exe scripts/wp2_calibrate.py
.venv312/Scripts/python.exe legacy_wp/scripts/wp3_train_gate.py
.venv312/Scripts/python.exe legacy_wp/scripts/wp3_score_tiles.py --split test
.venv312/Scripts/python.exe legacy_wp/scripts/wp4_measure_lod.py

# 4. catalogue et modèle de taille (le catalogue relit les images de data/)
.venv/Scripts/python.exe legacy_wp/scripts/wp6_build_catalogue.py
.venv/Scripts/python.exe legacy_wp/scripts/wp6_fit_size_model.py

# 5. simulations et vérifications (rapides)
.venv/Scripts/python.exe scripts/contact_windows.py
.venv/Scripts/python.exe legacy_wp/scripts/wp6_simulate_real.py
.venv/Scripts/python.exe legacy_wp/scripts/wp7_budget_sweep.py
.venv/Scripts/python.exe legacy_wp/scripts/wp8_queue_aware.py
.venv/Scripts/python.exe legacy_wp/scripts/wp5_optimality_gap.py --frac-steps 16
.venv/Scripts/python.exe legacy_wp/scripts/wp10_sensitivity.py

# 6. pipeline du papier
.venv/Scripts/python.exe scripts/paper_b0_b4.py                    # SIM
.venv312/Scripts/python.exe scripts/paper_b0_b4.py --source airbus --detector yolo

# 7. documents
.venv/Scripts/python.exe scripts/guide_figures.py
python scripts/build_full_guide.py

# 8. tableau de bord
.venv/Scripts/streamlit run dashboard/app.py
```

Le script `rerun_all.sh` relance toute la partie analyse d'un coup.

13. Ce qui est mesuré, ce qui est supposé, les limites

Chaque chiffre du projet porte une étiquette : **REAL** (mesuré sur Airbus), **SIM** (simulé), **LIT** (repris de la littérature), **TARGET** (objectif).

- **Le nuage est l'hypothèse la plus influente** : 52 % de la perte de navires vient du nuage, qui repose sur seulement 21 tuiles nuageuses réelles (Airbus est à 0,4 % de nuage). D'où la fourchette 0,40-0,82.
- **Les temps viennent d'un portable** (RTX 5060), pas du matériel de vol : « tourne à bord » n'est pas démontré, et l'énergie du papier est une estimation.
- **Les tuiles sont tirées au hasard**, pas en bandes orbitales réelles ; une seule station sol est modélisée.
- **Pipeline du papier** : jeu de données Airbus au lieu de xView, VisDrone, DOTA ou HRSID ; grandes scènes réelles = mosaïques 2×2 avec des jointures artificielles ; orbite du relais et débit ISL supposés.
- **Résultats négatifs publiés** : filtre classique insuffisant (0,645), INT8 plus lent, q30 coûte 4 points sur les petits navires, le paramètre aging_per_hour n'a aucun effet.
- **Corrections publiées** : l'avantage sur Phi-sat-2 est de +6,3 points, pas +16,7 comme annoncé au début, car la première comparaison était injuste.

14. Le tableau de bord

Un tableau de bord **au sol et en lecture seule** a été construit avec Streamlit et Plotly. Il lit les fichiers de code/results/ et ne recalcule rien : pour de nouveaux chiffres, on relance paper_b0_b4.py puis on clique sur « Recharger les fichiers ». Chaque page porte un bandeau SIM ou REAL.

Fichier	Rôle
dashboard/loaders.py	charge les CSV et le JSON ; choisit REAL si les fichiers existent, sinon SIM ; ne mélange jamais les deux
dashboard/app.py	l'interface : barre latérale (page, source, recharger) et 8 pages
scripts/paper_b0_b4.py	écrit en plus paper_links (fenêtres de liaison) et paper_scene (une scène exemple et ses détections) pour le tableau de bord
tests/test_dashboard.py	10 tests : chargements, et chaque page affichée par l'outil de test de Streamlit
scripts/dashboard_screenshots.py	les captures de cette section (Edge headless)

Lancer

```
cd code
.venv/Scripts/streamlit run dashboard/app.py
# puis http://localhost:8501   (ou ?page=Communication pour ouvrir une page)
```

Page	Ce qu'elle montre
Vue d'ensemble	indicateurs clés de B4, octets, qualité, énergie, tableau B0 → B4
Communication	chronologie sur 48 h : passages, fenêtres ISL, trajet de chaque paquet ; latence face à l'échéance
Compromis	curseur de seuil, courbes éq. 32-34, et l'éq. 20 résolue objet par objet
Scène	la scène, la grille SAHI et les cadres P1 / P2 / P3 ; vérité terrain en option
Ablations	ce que coûte chaque brique retirée, par rapport à B4 complet
Mesures réelles (fusion)	SAHI réel sur 24 bandes : 512 contre 768 px, relais, énergie
Résultats mesurés (WP)	les vrais chiffres Airbus : 0,685, ×557, mAP50, regret
Hypothèses	puissances, relais, SAHI : tout ce qui n'est pas mesuré



Page « Vue d'ensemble » — indicateurs clés de B4, octets, énergie, tableau B0 → B4. Capture du vrai tableau de bord, données SIM.



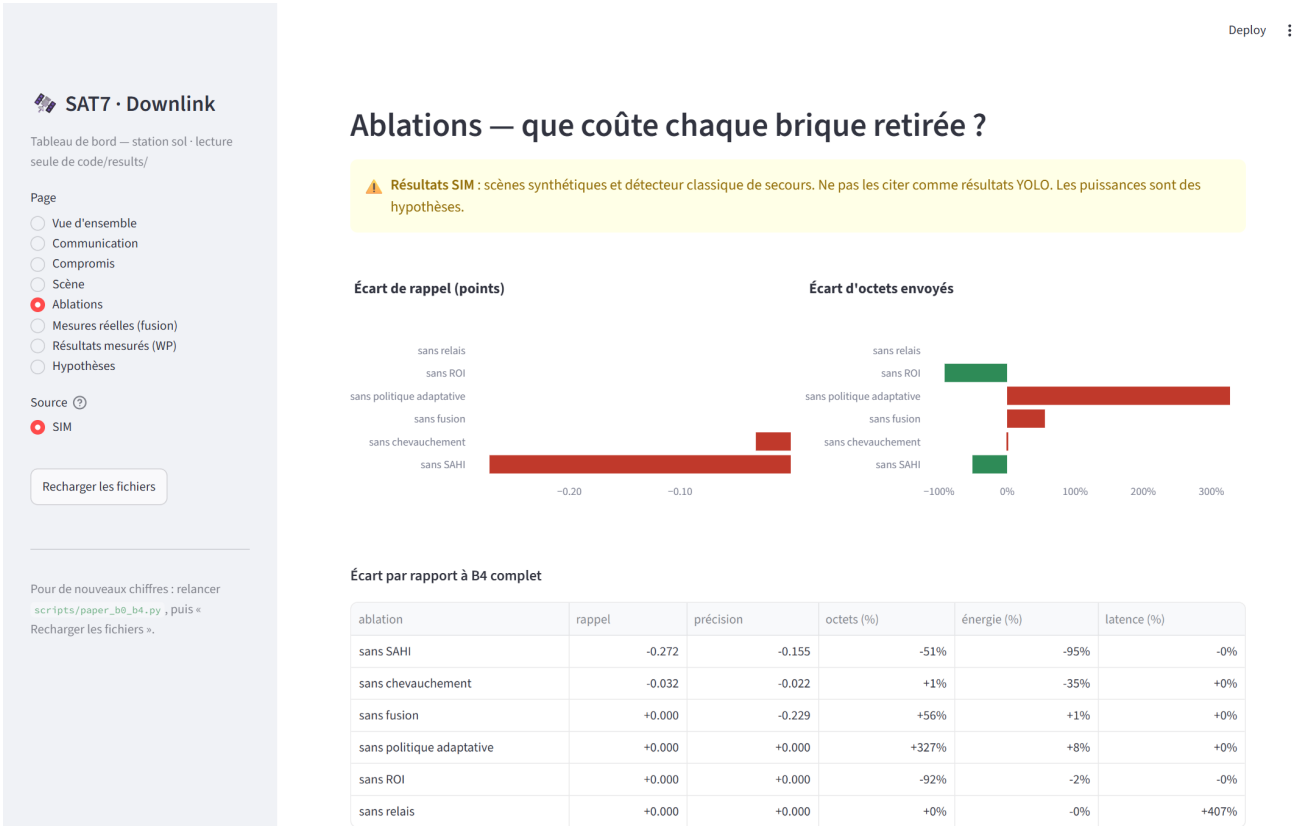
Page « Communication » — chronologie sur 48 h : passages, fenêtres ISL, trajet de chaque paquet ; latence face à l'échéance. Capture du vrai tableau de bord, données SIM.



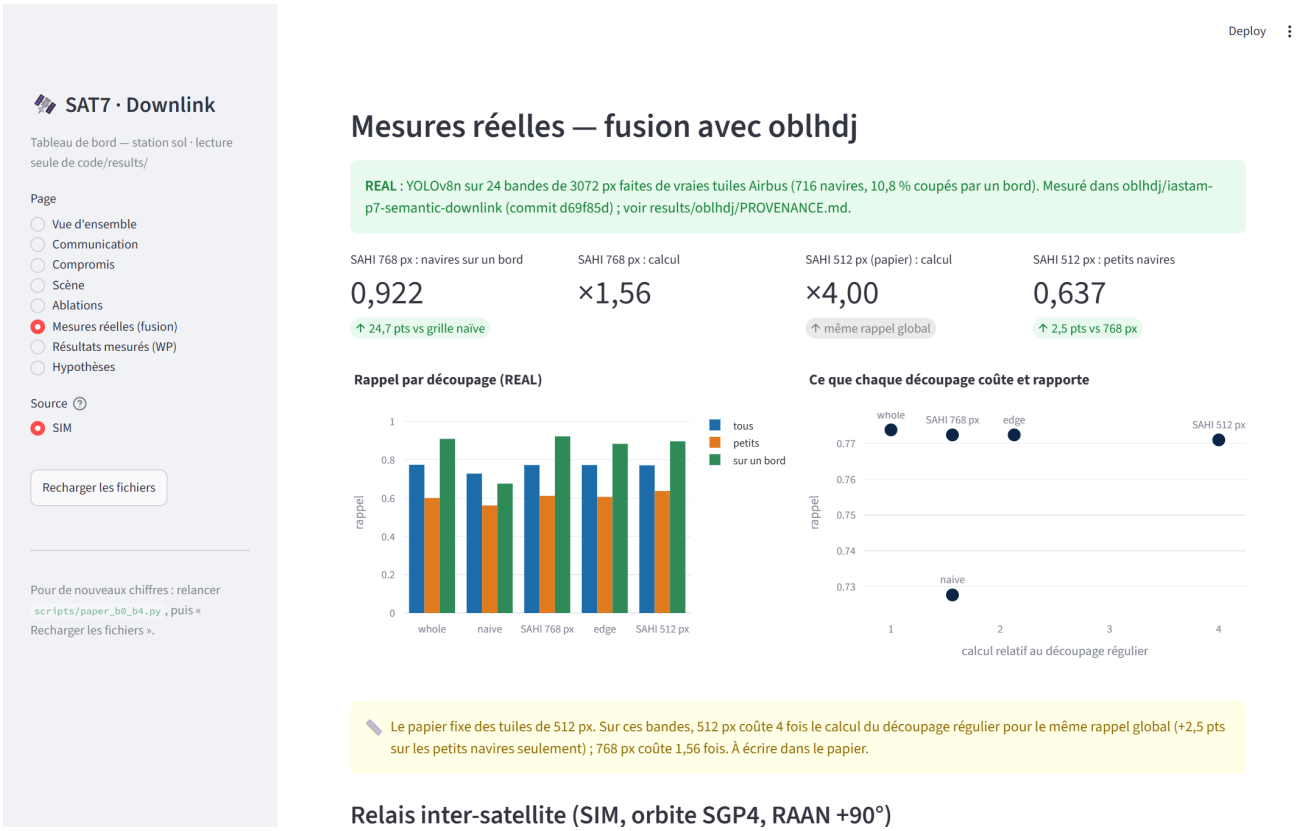
Page « Compromis » — curseur de seuil, courbes éq. 32-34, et l'éq. 20 résolue objet par objet. Capture du vrai tableau de bord, données SIM.



Page « Scène » — la scène, la grille SAHI et les cadres P1 / P2 / P3 ; vérité terrain en option. Capture du vrai tableau de bord, données SIM.



Page « Ablations » — ce que coûte chaque brique retirée, par rapport à B4 complet. Capture du vrai tableau de bord, données SIM.



Page « Mesures réelles (fusion) » — SAHI réel sur 24 bandes : 512 contre 768 px, relais, énergie. Capture du vrai tableau de bord, données SIM.

Deploy

SAT7 · Downlink

Tableau de bord — station sol · lecture seule de code/results/

Page
- Vue d'ensemble
- Communication
- Compromis
- Scène
- Ablations
- Mesures réelles (fusion)
- Résultats mesurés (WP)
- Hypothèses

Source
- SIM

Recharger les fichiers

Pour de nouveaux chiffres : relancer `scripts/paper_b0_b4.py` , puis « Recharger les fichiers ».

Résultats mesurés — pipeline WP sur tuiles Airbus

REAL : 5 320 tuiles Airbus de test, détecteur YOLOv8n entraîné, orbite SGP4.

Navires livrés	Réduction	Détecteur mAP50	Gate : rappel
0,685	×557	0,804	99,0 %

Stratégies (wp6_simulate_real.py)

stratégie	navires livrés	plafond (détectés)	efficacité liais
Bent pipe (images brutes)	0.002	0.841	0.0
Phi-sat-2, océan seul	0.518	0.518	
Phi-sat-2 équitable (+ côte)	0.622	0.622	
Nous : LoD sans mémoire	0.62	0.685	0.9
Nous : LoD + FIFO	0.685	0.685	
Nous : LoD + value-greedy	0.685	0.685	

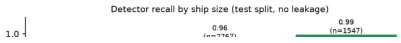
Rappel du détecteur par taille (wp1)

taille	navires	rappel
small (<32 px)	3859	0.739
medium (32-96 px)	2767	0.958
large (>96 px)	1547	0.994

Regret par politique (wp8) — plus petit = mieux

politique	pire	moyen
queue-aware (ours)	0.0058	0.0025
adaptive, fixed >=3	0.0123	0.0072
adaptive, fixed >=11	0.0168	0.0109
whole tile always	0.0253	0.0066
adaptive, fixed >=25	0.0269	0.0183
mosaic always	0.0656	0.0526





Page « Résultats mesurés (WP) » — les vrais chiffres Airbus : 0,685, ×557, mAP50, regret. Capture du vrai tableau de bord, données SIM.

Deploy

SAT7 · Downlink

Tableau de bord — station sol · lecture seule de code/results/

Page
- Vue d'ensemble
- Communication
- Compromis
- Scène
- Ablations
- Mesures réelles (fusion)
- Résultats mesurés (WP)
- Hypothèses

Source
- SIM

Recharger les fichiers

Pour de nouveaux chiffres : relancer `scripts/paper_b0_b4.py` , puis « Recharger les fichiers ».

Hypothèses — ce qui n'est pas mesuré

⚠ **Résultats SIM** : scènes synthétiques et détecteur classique de secours. Ne pas les citer comme résultats YOLO. Les puissances sont des hypothèses.

- **Source** : synthetic · **détecteur** : classic (fallback) · **scènes** : 24
- **Liaison** : cubesat_sband · **échéance** : 6.0 h

Puissances supposées (W)

	W
p_pre	5
p_tile	5
p_det	15
p_fusion	5
p_semantic	5
p_tx	12
p_isl	25
p_relay_gs	12

Relais inter-satellite (supposé)

	valeur
raan_offset_deg	60.0
isl_max_range_km	4500.0
isl_rate_bps	100000000.0
isl_efficiency	0.8
grazing_alt_km	100.0
step_s	20.0

Fenêtres de liaison sur l'horizon

	nombre
primary_passes	9
isl_windows	61
relay_passes	8

Configuration SAHI

	valeur
tile	512
overlap	0.2
fusion	nmm
match_metric	ios
match_thr	0.5

Page « Hypothèses » — puissances, relais, SAHI : tout ce qui n'est pas mesuré. Capture du vrai tableau de bord, données SIM.

15. La fusion avec le dépôt de l'ami (oblhdj)

Le dépôt *oblhdj/iastam-p7-semantic-downlink* part du même projet et a mesuré sur données réelles des parties du papier que ce dépôt simulait. La fusion garde votre pipeline du papier comme base et y ajoute ce que l'autre dépôt fait mieux (détail : `docs/MERGE_OBLHDJ.md`). Les fichiers importés sont dans `results/oblhdj/` avec leur provenance.

Repris	Où	Amélioré au passage
les 24 bandes réelles de 3 072 px (716 navires)	scenes.py, --source swaths	votre campagne reconstruit les mêmes scènes : comparaison directe
l'éq. 20 résolue objet par objet	joint.py, configuration B4+J	unités corrigées (×8) ; coûts = vrais octets et vrais temps d'encodage
plusieurs stations sol	orbit.py, relay.py, --stations	passages simultanés fusionnés (une seule radio)
chargeurs DOTA / HRSID	datasets.py, --source dota|coco	format Detection
temps réels de YOLO pour l'énergie	energy.detector_timing, --det-timing	E_det = inférences × temps mesuré
ses mesures REAL (wp15-wp23)	results/oblhdj/, page du tableau de bord	copiées telles quelles, avec leur provenance

SAHI mesuré sur les bandes réelles (REAL, YOLOv8n)

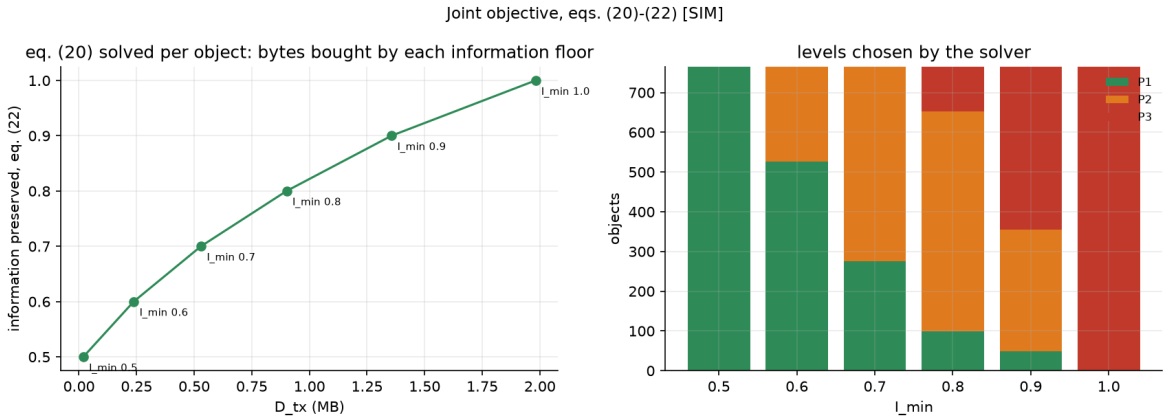
Découpage	Rappel	Petits navires	Navires sur un bord	Calcul
tuiles entières (oracle)	0,774	0,601	0,909	×1,00
grille naïve	0,728	0,561	0,675	×1,56
SAHI 768 px, 20 %	0,772	0,612	0,922	×1,56
SAHI 512 px (papier)	0,771	0,637	0,896	×4,00

SAHI récupère les navires coupés par un bord (0,922 contre 0,675). Mais aux 512 px du papier il coûte 4 fois le calcul pour le même rappel global : c'est un résultat à écrire dans le papier.

L'éq. 20 résolue objet par objet (SIM)

Chaque détection reçoit un niveau P0-P3 qui minimise αE + βD + γT sous les contraintes 21 et 22. Faire varier les poids ne change rien ici : énergie, octets et latence augmentent tous avec la taille d'un niveau. C'est le seuil d'information I_min qui décide :

I_min	Mo envoyés	Information (éq. 22)	P1 / P2 / P3	Énergie (J)
0,5	0,019	0,50	765 / 0 / 0	348
0,6	0,238	0,60	527 / 238 / 0	356
0,7	0,530	0,70	275 / 490 / 0	367
0,8	0,902	0,80	99 / 554 / 112	380
0,9	1,357	0,90	48 / 307 / 410	397
1,0	1,981	1,00	0 / 0 / 765	421



Octets achetés par chaque seuil d'information et niveaux choisis par le solveur [SIM].

Deux corrections trouvées pendant la fusion. (1) Dans l'autre dépôt, `sat7/energy.py` et `sat7/joint.py` divisent des octets par des bits/s : énergie et temps de transmission 8 fois trop bas (ses scripts wp17 et wp19 sont justes). (2) Dans ce guide, l'enregistrement de l'éq. 7 était annoncé à 18 octets ; il en fait 24, ce que le code a toujours produit.

Annexe — l'organisation des dossiers

Chemin	Contenu
README.md	présentation du projet, chiffres clés, comment lancer
reports/	15 rapports narratifs, un par lot de travail : à lire pour le « pourquoi »
docs/ARCHITECTURE.md	ce qui est construit, ce qui manque, classé
docs/STATUS.md	état actuel, points ouverts, environnements
docs/GLOSSARY.md	tous les termes et symboles (en anglais)
docs/PAPER_MAPPING.md	chaque équation du papier → sa fonction
paper/	les PDF : rapport technique, vue d'ensemble, guide poster, ces guides
code/sat7/	la bibliothèque (section 7)
code/scripts/	un script par étape (sections 5 et 6)
code/dashboard/	le tableau de bord Streamlit (section 14)
code/legacy_wp/	le pipeline WP archivé : wplib/, scripts/, tests/ (sections 4-5)
code/tests/	les tests du papier, du tableau de bord et du socle (section 11)
code/results/	toutes les sorties ; les CSV font foi ; journaux dans results/logs/
code/data/, code/runs/, code/models/	dataset et poids : hors git, reconstruits par wp0-wp3
code/rerun_all.sh	relance toute l'analyse

Le développement a été assisté par Claude (Anthropic). Chaque expérience, chiffre et conclusion a été spécifié, exécuté et relu par l'équipe.